eve_rl Multi-Mesh Generalization Report

DualDeviceNav endovascular-navigation RL stack — three code-grounded audits (mesh coupling, observation mesh-invariance, privileged-critic plumbing) and the multi-mesh generalization design

Compiled 8 July 2026

Sources: 6.md – 9.md (d:\Arjun)

Project: workspace/neve — eve_rl SAC/AWAC training stack

Contents

PART 1 OF 4

Mesh-Coupling Audit — "neve" RL vascular-navigation codebase

Agent: Mesh-coupling hardcoding audit · Source file: 6.md

Bottom line: The heavy RL machinery (reward shaping, path state-machine, observations) was already **de-hardcoded** in "Plan v12" and is now driven almost entirely from the loaded vessel tree + planned path at reset — it is portable to any mesh **whose centerlines reuse the same naming convention** (named daughters `LCCA/RCCA/RVA/LVA` + numbered trunk/bridge curves). The real coupling lives in three places: (1) the branch **naming convention** (tag strings + `.mrk` templates), (2) a handful of **absolute-coordinate / registration** constants (insertion z≈345, insertion point, `rotation_yzx_deg=[90,-90,0]`), and (3) the **heuristic demo generators**, which are junction-relative in mechanism but carry mesh-tuned windows, fixed vessel-CS direction vectors, and topology token-chains. A config-swappable mesh path (`DualDeviceNavCustom`) and a per-episode geometry-randomization precedent (`ArchVariety` / `AorticArchRandom`) both already exist.

---

1. Inventory of mesh-specific hardcodings (file:line → verdict)

`eve/eve/util/pathcontext.py` — almost fully portable

Location	Item	Verdict
`:57` `_DAUGHTER_TAGS = ("RCCA","LCCA","RVA","LVA")`	daughter name tags	**Portable if naming preserved.** Pure string tokens; work on any mesh whose daughter centerlines contain these substrings.
`:60-80` `_physical_named_tag` regex `\((\d+)\)` → `(2),(0)` =trunk, `(11)` =bridge	numbered-curve topology map	**Semi-portable.** Only assigns the coarse `trunk` / `bridge` labels; the *daughter* classification (`:83-132` `classify_physical_branch`) keys off the named tags, so wrong numbers merely collapse trunk/bridge to `"other"` — daughter identity still correct.
`:141` `ON_PATH_TOLERANCE_MM=3.0`, `:146` `CROSS_TRACK_TOLERANCE_MM=10.0`, `:154-158` `K_RADIUS=1.5`, `MIN_TOLERANCE_MM`, `DEFAULT/MIN/MAX_RADIUS`, `:163` `COMMIT_HYSTERESIS_MM=10.0`, `:171` `WRONG_COMMIT_ARC_MM=10.0`	radius/mm-based thresholds	**Portable** (mm- and local-radius-relative, as the task predicted).
`:406-872` `reset()`, `_build_branch_index`, `_build_branching_graph`, `_build_path_branch_sequence`, `_build_entry_points`	ALL path geometry (polyline, cumulative arclength, junction arclengths, daughter arcs, branch sequence, radii, BP graph)	**Portable — derived at reset** from `pathfinder.path_points_vessel_cs` + `vessel_tree.branches` / `branching_points`. No baked arclengths or coordinates.

Net: pathcontext has **no baked magic arclengths and no absolute coordinates**; only the 4 daughter tag strings + the numbered-curve coarse map are naming-dependent.

`training _scripts/util/env5.py` — mostly de-hardcoded (Plan v12)

Location	Item	Verdict
`:37` `DAUGHTER_TAGS`, `:40-49` `_branch_short_from_name`, `:562` `("LCCA","LVA","RCCA","RVA")`, `:1503` `("RCCA","RVA","LCCA","LVA")`, `:1005/:1053` OST predicate tag list	daughter tag matching	**Portable if naming preserved.**
`:1465-1508` `_classify_branch_segment`	reward branch-class (trunk/bridge/daughter)	**Portable — structural, NOT numbered.** Explicitly rewritten in Plan v12: trunk = first on-path branch (`pc._trunk_branch_idx`), target = last on-path branch, bridge = any other on-path branch. No `(2)/(0)/(11)` literals.
`:1548-1601` `_compute_path_segment_step_reward`, `:1524-1538 _trunk_end_arc`	per-step reward	**Portable.** Trunk interpolation uses `proj_s / trunk_end_arc` where `trunk_end_arc = get_path_junctions()[0]` (runtime). The "z=345" in comments (`:1456,:1526,:1562`) is descriptive only — the math is arclength-relative.
`:940-943` regex `\(0\)` → `\(11\)` (pre-bif11 checkpoint), `:976` `"(11)" in prev_n and "RVA" in next_n` (RVA checkpoint)	numbered topology	**Non-portable BUT env-var-gated tooling only** (`PRE_BIF11_CHECKPOINT_DIR`, `RVA_CHECKPOINT_DIR`). Off in normal training; used only for checkpoint-harvest scripts.
`:1827-1832` `DAUGHTER_CENTERLINE_TEMPLATES = {"RCCA":"Centerline curve - RCCA.mrk", ...}`, `:1826` `DAUGHTER_SHORT_NAMES`	exact `.mrk` centerline filenames	**Non-portable-literal, portable-if-naming-preserved.** MultiTargetEnv5 samples per-daughter targets by these exact names.

`training _scripts/DualDeviceNav_train.py`

Location	Item	Verdict
`:18` `RESULTS_FOLDER="../mesh_ben"`	output path label	Cosmetic; non-portable label only.
`:21-26` `TARGET_BRANCHES` = 4 `.mrk` names	default targets	**Portable if naming preserved** (overridable via `--target_branches`).
`:60` `EVAL_SEEDS` (98 seeds)	eval target sampling	**Single-mesh assumption.** Seeds index `CenterlineRandom` on whatever mesh is loaded; a held-out eval mesh just needs its own `DualDeviceNavCustom` intervention — the seed list is mesh-agnostic but the target *pool* is one mesh's.
`:360,:433` `DualDeviceNav(insertion_z=args.insertion_z)`	mesh source	**The single mesh binding.** No mesh path in the train script — the mesh is entirely inside `DualDeviceNav`.
`:1091-1100` `--insertion_z` (help cites ~345)	insertion anchor	**Non-portable value** (~345 is this mesh's "40 mm below bif2"); it's a CLI arg, re-derive per mesh.

`eve_bench/eve_bench/dualdevicenav.py` — the mesh definition

Location	Item	Verdict
`:12-13` `DATA_DIR=data/dualdevicenav`, `:24-25` `vessel_architecture_{collision,visual}.obj`, `:27` `Centrelines_comb`	hardcoded mesh + centerline paths	**Non-portable (by design in `DualDeviceNav`).**
`:87-88` insertion `[65.0,-5.0,35.0]`, direction `[-1.0,0.0,1.0]`	fallback femoral entry	**Non-portable absolute coordinates.**
`:95` `rotation_yzx_deg=[90,-90,0]`	mesh→SOFA registration	**Non-portable registration constant.**
`:42-85` `insertion_z` trunk detection (comments cite trunk "(2)", z-range `[79.4, 392.0]`)	z≈392 = bif2	**Non-portable.** Confirms z=345 insertion is ~40 mm below bif2 on THIS mesh.
`:148-153` target `.mrk` names; `:182` `(y,-z,-x)` axis remap	targets + coordinate convention	Targets portable-if-named; the remap is a Slicer-export convention (portable across Slicer-exported meshes).
`:231-408` `DualDeviceNavCustom`	config-swappable mesh loader	**This is the portability escape hatch** (see §2).

---

2. Mesh-loading pathway — what a "mesh" is

Call chain: `DualDeviceNav_train.py:360` → `DualDeviceNav.__init__` (hardcoded OBJ + `Centrelines_comb`) → `eve.intervention.vesseltree.FromMesh(mesh, insertion, insertion_direction, branch_list=branches, rotation_yzx_deg=[90,-90,0], visu_mesh=...)` (`dualdevicenav.py:90`) → `FromMesh` loads/rotates/scales the

file via PyVista and stores `mesh_path` → at every episode `MonoPlaneStatic.reset()` (`monoplanestatic.py:146-162`) calls `simulation.reset(mesh_path=self.vessel_tree.mesh_path, ...)`.

What defines a mesh: - **Geometry:** a PyVista-readable file (`.obj/.stl/.vtk/.vtp/.vtu/.ply`) — `FromMesh` is fully mesh-agnostic (`meshing.py load_mesh = pv.read`). - **Centerlines/topology/radii:** a folder of `Centerline curve *.mrk.json` files (`load_branches`, `dualdevicenav.py:199-228`) — named daughters (`-LCCA.mrk`) as branch 0-style, numbered `(N)` curves as trunk/bridge. Radii come from the JSON `measurements` (MISR). **This naming is the load-bearing metadata.** - **Registration:** `rotation_yzx_deg`, insertion point + direction.

Config-swappable path already exists: `DualDeviceNavCustom(mesh_folder, model_name, insertion_point, rotation_yzx_deg, fluoroscopy_rot_zx)` (`dualdevicenav.py:231-408`) resolves `{model_name}_collision.obj`/`_visual.obj` + a `Centrelines/` folder, auto-derives insertion from branch 0, and auto-derives targets as "all branches except first." The `vmr_processing_tools/` pipeline (`create_dualdevicenav_format.py`) produces exactly this layout from VMR VTP models (10× scale, decimate, VMTK centerlines with radius).

⚠️ **Naming caveat for new meshes** (critical, cross-cutting): the VMR pipeline by default names centerlines `Centerline curve - <model_name>.mrk` or lowercase per-vessel (`aorta`, `lcca`). The whole tag-matching stack (pathcontext, env5, heuristics, `TARGET_BRANCHES`) matches **uppercase** `LCCA/RCCA/RVA/LVA` plus numbered `(2)/(0)/(11)/(18)/(19)` curves. New meshes must be authored/renamed to reproduce this convention, or the tag matching silently falls to `"other"`. Also note the folder-name mismatch: `DualDeviceNav` reads `Centrelines_comb`, `DualDeviceNavCustom` reads `Centrelines`.

Per-episode geometry precedent: `ArchVariety` (`archvariety.py`) → `AorticArchRandom(episodes_between_change=N)` re-randomizes the arch every N episodes (`_randomize_vessel()`), synthesizing the mesh from centerlines (no file). It uses plain `BenchEnv` (env v1, PathDelta reward) — **not** env5 — and 10× larger `HEATUP_STEPS` (5e5 vs 2e4). A file-based `VMR` loader (`vmr.py`) also exists, selecting a model by string and deriving insertion from a named vessel.

---

3. Feasibility: per-worker mesh vs per-episode swap

Worker cloning mechanism: ONE master `env_train` is deep-copied per worker at a single site — `synchron.py:702 deepcopy(self.env_train)` inside `_create_worker_agent(i)` (called for `i in range(n_worker)`), then pickled into each `spawn`ed subprocess (`singelagentprocess.py:407-432`). The clone happens **pre-reset**, when SOFA is not yet imported (`sofabeamadapter.py:256-259` defers `import Sofa` to `reset()`), so each worker builds its own independent SOFA scene in its own process. Network dims are frozen from the one master env's spaces (`agent.py:175-180`). Worker id `i` is already in scope at the clone site; a per-worker seed already flows (`singelagentprocess.py:136-145`) but reaches the *agent*, never the env.

SOFA lifecycle: `SofaBeamAdapter.reset()` rebuilds the scene (unload → `_load_plugins` → `_basic_setup` → `_add_vessel_tree(mesh_path)` reloads the `.obj` via `MeshObjLoader` → `_add_devices` rebuilds the beam graph → `Simulation.init`) **only when** `root is None` OR `mesh_path`/insertion/coords differ from cached (`sofabeamadapter.py:261-270`). Because `FromMesh.reset()` is a no-op and `mesh_path` is fixed at construction, after episode 1 the guard is always "unchanged" and reset falls through to a cheap `_update_properties()` + separate `reset_devices()`. **So: build-once-per-worker, cheap-reset-per-episode.**

Per-worker fixed mesh — **RECOMMENDED, small change, ~zero extra runtime cost**

Fits the architecture perfectly: each worker already builds its scene exactly once at startup and never rebuilds. Giving worker `i` a different mesh costs one scene build per worker at startup (same startup cost paid today), zero per-episode cost. Concrete changes: 1. `synchron.py:_create_worker_agent(i)` — replace `deepcopy(self.env_train)` (line 702) with `self.env_train_factory(i)` or `deepcopy(self.env_train_list[i % k])`. Restart path (`_restart_worker_agent`, reuses `agent_id`) is then correct for free. 2. `synchron.py __init__` + `agent.py:341-351 (BenchAgentSynchron)` — accept and forward a list/factory instead of a single `env_train`. 3. `DualDeviceNav_train.py:360` — build `k` interventions via `DualDeviceNavCustom(mesh_folder=mesh_k, ...)` and pass the list/factory. 4. Route per-mesh target-branch schedules per worker (`synchron.py:458-469 _split_schedule` currently assumes homogeneous workers).

Per-episode mesh swap — **mechanically supported but expensive + API-awkward**

`AorticArchRandom` proves it works: vary `vessel_tree.mesh_path` and the next `reset()` trips the rebuild guard. But (a) each swap is a **full scene rebuild** (`Simulation.init` is the known slow/hang-prone call; no hot-swap — the mesh is a `MeshObjLoader` node), and (b) the deeper blocker is that mesh/centerlines/targets/insertion/coord-space are all baked at **construction** in `DualDeviceNav` / `FromMesh` / `CenterlineRandom`, and `FromMesh.reset()` cannot change them — so a true per-episode swap means rebuilding the whole intervention object, which the `reset(seed, options)` API can't express. Would require either adopting the `AorticArchRandom` pattern (a multi-mesh `VesselTree` subclass whose `reset()` re-points `mesh_path` among a pool + rebuilds `branch_list`/coord-space) or a custom wrapper. No repo timing exists, but stepping is 0.5–0.8 s/step (episodes run minutes), so a seconds-scale rebuild per episode is noticeable, not dominant.

Recommendation: per-worker fixed meshes for training (cheap, aligned with architecture); reserve per-episode swap only if you need every worker to see every mesh.

---

4. Heuristic / demo portability

Heuristics generate the AWAC demo data; all phase anchors are looked up **at runtime** from `get_path_junctions()` and matched by branch-name substring — so the anchor *positions* are geometry-relative (portable), but three things are baked in.

- `heuristic_policy.py` **(base) +** `heuristic_controller.py`**:** fully geometry-relative — path membership, `d_corr`, arc-past-junction all from `_path_context`; only device/action constants (retract range, fold brake, `K=5` node spacing). **Zero retuning.**
- `heuristic_policy_rcca.py` **/** `_rva.py`**:** junction token chains `(2)→(0)→(11)→RCCA/RVA` (`:271-273` / `:270-272`) — the numbered `(11)` bridge and 3-junction chain are topology-specific; fixed ±z Phase A/B `TARGET_DIR` vectors (orientation-coupled, but mostly speed-modulators); cavity split `s < jn_arc + 5.0` tuned to this mesh's measured 5 mm merged RVA/RCCA cavity; `PHASE_C_VARIANTS` grid-tuned on this mesh. **Retune: re-verify the token chain, re-fit ±z dirs, re-run the C0–C8 variant grid (harness exists), sign smoke-test.**
- `heuristic_policy_lcca.py`**:** assumes bridge-less `(0)→LCCA` (`:262-263`); Phase A `[0,0,-1]` dir; 25 mm waypoint depth + windows sized to this mesh's 12 mm bif2 cavity. **Retune ~4 constants + 1 dir vector + topology check.**

- `heuristic_policy_lva.py`: lowest constant count, **highest structural fragility.** Token `(18)→LVA`; the strategy itself relies on *this mesh's* arch passively funneling the wire into `(18)` with no active Phase-A steering (`:3-11`), and an off-path override justified by LCCA/LVA convergence flicker to `(19)`. On a differently-curved arch these premises can fail with nothing to retune — may need a new active trunk-top phase.

Effort estimate: base 0; RCCA/RVA a constants re-grid + sign smoke-test; LCCA that plus window/waypoint re-fit; LVA a strategy re-validation. Failure mode is graceful (unmatched token → phase disables → falls back to centerline follower), not a crash. **Demos must be regenerated per mesh regardless**, since arclengths/coords differ.

---

5. Shortest path to "3 train meshes + 1 held-out eval mesh"

Prereq (do first): author the 3+1 meshes with the existing naming convention. Run `vmr_processing_tools/create_dualdevicenav_format.py` per model, then **rename centerlines** to the uppercase daughter tags (`Centerline curve - LCCA/RCCA/RVA/LVA.mrk.json`) + numbered trunk/bridge curves matching the current topology (`(2)` trunk, `(0)/(11)/(18)/(19)` bridges). Verify radius data (`verify_radius_data.py`). This is the single biggest prerequisite — without consistent naming, tag-matching silently degrades.

Per-file change list:

1. `eve_bench/eve_bench/dualdevicenav.py` — none required if you use `DualDeviceNavCustom`; optionally reconcile its `Centrelines` vs `Centrelines_comb` folder name and confirm it can auto-derive or accept the 4 named targets (it currently auto-derives "all but first" — pass explicit target branch names for parity).
2. `eve_rl/eve_rl/agent/synchron.py` — accept `env_train_list` / `env_train_factory`; change `:702 deepcopy(self.env_train)` → `self.env_train_factory(i)`. Guard checkpoint save/load (`:777 cp["env_train"]`) for the multi-env case (serialize per-worker mesh id, or accept single-env round-trip loss).
3. `training _scripts/util/agent.py` (`BenchAgentSynchron`, `:341-351`) — forward the list/factory; keep deriving obs/action dims from mesh 0 (all meshes must share obs/action shapes — same observation feature counts).
4. `training _scripts/DualDeviceNav_train.py` — build 3 training interventions via `DualDeviceNavCustom(mesh_folder=mesh_k, insertion_point=..., rotation_yzx_deg=...)`, assign round-robin across the 16 workers (mesh `k = i % 3`); build the **held-out** 4th mesh as a separate `env_eval` (keep eval on the one held-out mesh rather than splitting eval seeds across per-worker meshes — see synchron `_split`). Add a `--train_mesh_folders` / `--eval_mesh_folder` CLI. Re-derive `--insertion_z` per mesh.
5. `training _scripts/util/heuristic_policy_*.py` — if seeding AWAC demos, regenerate per training mesh; retune per §4 (base 0, RCCA/RVA re-grid, LCCA constants, LVA strategy re-validation). Route the right `--heuristic_factory` + per-mesh target schedule per worker.
6. **No change needed:** `pathcontext.py` (fully runtime-derived), `env5.py` reward/state-machine core (`_classify_branch_segment` is structural), `FromMesh` / `SofaBeamAdapter` (mesh-agnostic; per-worker build-once already correct).

Watch-outs: (a) all meshes must yield identical observation/action space shapes (frozen from master env); (b) eval semantics — keep the held-out mesh as a dedicated eval env, not split across workers; (c) coordinate-space normalization + insertion + targets are per-mesh (baked at construction), so per-mesh options must be routed per worker; (d) `ArchVariety` / `AorticArchRandom` is a ready reference if you later want per-episode variety instead of per-worker fixed meshes.

Key files (all absolute): `d:\Arjun\workspace\neve\eve_bench\eve_bench\dualdevicenav.py`, `d:\Arjun\workspace\neve\eve\eve\intervention\vesseltree\frommesh.py`, `d:\Arjun\workspace\neve\eve\eve\intervention\monoplanestatic.py`, `d:\Arjun\workspace\neve\eve\eve\intervention\simulation\sofabeamadapter.py`, `d:\Arjun\workspace\neve\eve_rl\eve_rl\agent\synchron.py`, `d:\Arjun\workspace\neve\eve_rl\eve_rl\agent\singelagentprocess.py`, `d:\Arjun\workspace\neve\training _scripts\DualDeviceNav_train.py`, `d:\Arjun\workspace\neve\training _scripts\util\env5.py`, `d:\Arjun\workspace\neve\eve\eve\util\pathcontext.py`, `d:\Arjun\workspace\neve\training _scripts\util\heuristic_policy_{lcca,rcca,rva,lva}.py`.

PART 2 OF 4

Mesh-Invariance Audit of the Observation Stack (env5 / LocalGuidance)

Agent: Obs mesh-invariance audit · Source file: 7.md

0. The full observation as the policy sees it

Built in `BenchEnv5.__init__` — `d:\Arjun\workspace\neve\training _scripts\util\env5.py:240-301` (note the directory really is `training _scripts` with a space). `ObsDict` (`d:\Arjun\workspace\neve\eve\eve\observation\obsdict.py`) preserves insertion order, and the eve_rl embedder flattens it in that order (confirmed by the `obs[40:42]` comment in `target2d.py:29`):

Flat slice	Key	Component chain	Dims	Frame	Normalization
0:40	`tracking`	`Tracking2D(n_points=10, resolution=15)` → `NormalizeTracking2DEpisode` → `Memory(n_steps=2, FILL)`	2×10×2 = 40	**absolute 2D image coords** (vessel CS rotated by fixed C-arm `image_rot_zx=[20,5]`, Y dropped)	affine map of the **whole-mesh centerline bounding box (±10%)** to [-1,1], per axis
40:42	`target`	`Target2D(target_coord3d=frozen grader coord)` → `NormalizeTracking2DEpisode`	2	**absolute 2D image coords**	same mesh-bbox affine
42:46	`last_action`	`LastAction` → `Normalize`	2×2 = 4	action space (gw/cath × trans/rot velocity)	static bounds low=[[-10,-1.5],[-10,-1.5]], high=[[30,1.5],[30,1.5]] (so trans=0 maps to −0.5)
46:76	`guidance`	`LocalGuidance` (raw, no wrapper, single frame — **no Memory**)	30	mixed (see table)	per-feature clips (mm constants)
76:78	`inserted_lengths`	`InsertionLengths` → `Normalize`	2	scalar mm	by device max length = 900 mm each (`monoplanestatic.py:64-65`)

Coordinate frames: "vessel CS" = the mesh's own mm frame; "tracking3d" = vessel CS rotated by the fixed C-arm rotation (`coordtransform.py:27-47`, `image_center=[0,0,0]` for this intervention so it is pure rotation); "2D image" = tracking3d with Y deleted (`coordtransform.py:65-66`).

The per-episode normalization reference (`NormalizeTracking2DEpisode`, `eve\eve\observation\wrapper\normalizetracking2depisode.py:20-29`) is `fluoroscopy.tracking2d_space_episode` = **min/max over ALL centerline coordinates of the entire vessel tree**, padded by a sign-dependent ±10% (`eve\eve\intervention\fluoroscopy\trackingonly.py:53-76`). On a static mesh this is constant every episode. **Its reference quantities are exactly the mesh's size, aspect ratio and absolute placement** — and because x and z are scaled by different factors, the normalization is anisotropic (angles/curvatures of the wire shape get distorted by a per-mesh-different affine).

1. LocalGuidance — complete 30-dim feature inventory

Source: `d:\Arjun\workspace\neve\eve\eve\observation\localguidance.py` (space at :236-248, assembly at :466-500). Verdict key: **INV** = honestly transferable local/path-relative geometry; **SCALE** = transferable in mm but sensitive to vessel-size distribution shift; **ROUTE-PARAM** = path-relative but parameterized per-route (supports positional indexing / memorization); **MESH** = mesh/anatomy-identity information; **DEAD** = constant in RL runs.

Idx	Name	Computation (file:line)	Frame	Normalization / bound	Verdict
0	`d_rem_norm`	`(total_len - proj.s)/total_len` (:308-309)	path arclength	÷ this route's total length, [0,1]	**ROUTE-PARAM** — a per-(mesh,target) progress index; "at f=0.4 turn left" is memorizable, and the same feature step means different mm on different meshes
1	`cross_track_dist`	perpendicular tip→polyline distance from `project_onto_polyline` (:311-312, `polyline.py:82-144`)	tip-local, mm	clip 50 ÷ 50, [0,1]	**INV** (mm-absolute → SCALE if mesh sizes differ)
2	`tangent_x_2d`	planned-path segment tangent at projection, rotated to tracking3d, Y dropped, renormalized (:276-283, :314-320)	2D image axes	unit, [-1,1]	**INV** — local path direction; world-axes expression is fine because the C-arm is fixed and identical across meshes
3	`tangent_z_2d`	as above	2D image axes	unit	**INV**
4	`heading_error`	signed angle(device dir from tracking[0]−tracking[1], path tangent) in tracking3d, sign from image-plane Y cross product (:506-544)	relative (device vs path)	÷π, [-1,1]	**INV** — the cleanest transferable feature
5	`curvature_ahead`	max discrete curvature over next 20 mm (`_LOOKAHEAD_MM`) (:546-558, `polyline.py:53-79`)	path-local	÷10 rad/mm (heavily under-scaled: real values ~0.1–1)	**INV**
6	`dist_to_bifurc`	arclength to next branching point (:560-568)	path arclength, mm	clip 200 ÷ 200	**INV**/SCALE
7	`on_correct_branch`	state-machine `is_on_correct_branch()` via path_context (:331-337)	topological	{0,1}	**INV**
8	`dist_correct_entry`	**graph-routed** retrace+reroute distance to next correct daughter entry, `get_routed_d_corr_to_next_daughter_entry` (:345-350, `pathcontext.py:1334-1409`)	centerline graph, mm	clip 200 ÷ 200	**INV**/SCALE
9	`correct_entry_dir_x`	unit vector tip→daughter-entry interior point, rotated to tracking3d, 2D (`_entry_direction` :173-194)	2D image axes, tip-relative	unit	**INV**

Idx	Name	Computation (file:line)	Frame	Normalization / bound	Verdict
10	`correct_entry_dir_z`	as above	2D image axes	unit	**INV**
11	`arc_to_next_daughter_norm`	`get_arclength_to_next_daughter_entry()` (:360-376, `pathcontext.py:1434-1449`); ∞→1.0	path arclength, mm	clip 100 ÷ 100	**INV**/SCALE
12	`arc_past_last_daughter_norm`	`get_arclength_past_last_daughter_entry()` (:369-377, `pathcontext.py:1451-1467`)	path arclength, mm	clip 100 ÷ 100	**INV**/SCALE
13	`phase_default`	one-hot from `intervention._heur_rva_phase`, mirrored by env5 (:379-384, env5.py:789-798)	n/a	{0,1}	**DEAD in RL** (always "default" = 1 unless a heuristic policy writes phases); when active, phase tokens (`lcca_jn`, `trunk_top`…) are **MESH/anatomy-specific**
14	`phase_trunk_top`	as above	n/a	{0,1}	DEAD / MESH
15	`phase_bridge`	as above	n/a	{0,1}	DEAD / MESH
16	`phase_daughter`	as above	n/a	{0,1}	DEAD / MESH
17	`bend_hat_x_2d`	J-tip second-difference `p0+p2-2p1` of first 3 tracked points, perpendicular to tip tangent, projected to image 2D (`_compute_bend_hat_2d` :132-170)	tip-local → image axes	unit	**INV**
18	`bend_hat_z_2d`	as above		unit	**INV**
19	`off_arc_since_divergence_norm`	arc traveled along wrong branch since off-path flip (`pathcontext.py:1314-1332`)	wrong-branch arclength, mm	clip 50 ÷ 50	**INV**

Idx	Name	Computation (file:line)	Frame	Normalization / bound	Verdict
20	`off_branch_steps_norm`	env counter mirror `_env_off_branch_steps` ÷ 50 (:402-406, env5.py:806-812)	step count	[0,1]	**INV**
21	`fold_stall_count_norm`	env counter mirror `_env_fold_stall_count` ÷ 20 (:403-407)	step count	[0,1]	**INV** (but quantized/thresholded — see C)
22	`episode_step_norm`	`_env_episode_step / _env_max_steps` (:404-408)	time	[0,1]	**ROUTE-PARAM-ish** — with a deterministic start it is an open-loop schedule index
23	`forks_correct_norm`	`_n_correct_commits ÷ len(_path_branch_sequence_with_junctions)` (:410-424)	topological count	per-route denominator	**INV**, but the per-route quantization step (1/2 vs 1/3…) fingerprints the route
24	`forks_wrong_norm`	as above (:425)			same
25	`is_in_trunk`	`_current_branch_idx == _trunk_branch_idx` (:442-459, `pathcontext.py:453-465`)	topological	{0,1}	**INV**
26	`is_on_target_daughter`	`== _target_daughter_branch_idx` (:463-464)	topological	{0,1}	**INV**
27	`is_in_a_wrong_branch`	`not _on_planned_path` (:452-454)	topological	{0,1}	**INV**
28	`is_at_ostium`	`== _ostium_branch_idx` (last on-path bridge before target) (:461-462)	topological	{0,1}	**INV**
29	`wrong_recovery_dist_norm`	off-path only: reuses routed `d_correct`, clip 200 ÷ 200 (:454-458)	graph, mm	[0,1]	**INV**/SCALE

Docstring says "28-dim" (:3, :198) — the array is actually 30 (:234, :247). Cosmetic, but worth fixing before adding features.

Summary: LocalGuidance is largely a *relative/topological* observation and is mostly transferable. The only genuinely dangerous entries inside it are 0 (route-parametric progress index), 13-16 (dead one-hots with anatomy-bound semantics), and mildly 22-24. Absolute position never appears in LocalGuidance — the absolute-position leaks live in the *other* components.

A. Which features support mesh memorization vs. honest transfer

Memorization channels, ranked by severity:

1. `tracking` **(40 dims, flat 0:40) — the dominant leak.** Absolute image-plane wire shape normalized by the mesh's own bounding box (`trackingonly.py:53-76`). Every frame tells the policy *where in mesh A's bbox the wire is*, in coordinates that are a fingerprint of mesh A (its bbox aspect + the characteristic centerline layout). On one mesh, this alone supports open-loop replay: absolute tip position ≈ progress state.
2. `target` **(flat 40:42).** Absolute bbox-normalized 2D target coordinate (`target2d.py:50-65` + same normalizer). On a single mesh the 4 daughters occupy 4 recognizable coordinate clusters — the policy can key its whole route off this constant instead of reading path guidance. Across meshes it doubles as a mesh-identity cue.
3. `inserted_lengths` **(flat 76:78).** Absolute pushed length ÷ 900 (`insertionlengths.py:16-22`). With a fixed insertion point (z=345), mesh landmarks sit at fixed insertion depths (the code itself relies on this: "checkpoints place the wire at ~372 mm insertion past bif1", env5.py:692-694). This is a per-mesh odometer — excellent for memorized action schedules, wrong across meshes.
4. `guidance[0] d_rem_norm` — route-parametric index (see table).
5. `guidance[22] episode_step_norm` — time odometer.
6. `guidance[13:17]` **phase one-hots** — dead in RL; anatomy-bound if ever active.
7. `guidance[23:25]` **quantization** — reveals route junction count (minor).

Honestly transferable: `last_action`; guidance 1-12, 17-21, 25-29 (all local mm-scale, relative-direction, or topological-state features); the *shape* content of `tracking` if re-expressed tip-relative (see D). One caveat that applies to all "2D image axes" direction features (2-3, 9-10, 17-18): they are only mesh-invariant if all meshes are loaded in a consistent anatomical orientation with the same `image_rot_zx=[20,5]` (`dualdevicenav.py:137-142`; `DualDeviceNavCustom` lets this vary — keep it fixed, or the direction features silently change meaning).

Concrete changes: - `tracking`: replace `NormalizeTracking2DEpisode` with tip-relative + fixed-scale normalization (D below). - `target`: delete the absolute coordinate. Its navigational content is already carried by guidance 0/8/9/10. If you want an explicit target cue, add tip→target offset clipped to ±50 mm ÷ 50 (2 dims, computable from `fluoroscopy.tracking3d[0]` and the frozen `target_coord3d`, both already at hand in `_CoordTargetReachedReward.step`, env5.py:153-163) — far targets saturate, path guidance handles routing. - `inserted_lengths`: replace absolute lengths with **(a)** gw-slack = `inserted_gw − proj.s` (see C — this is the physically meaningful, mesh-relative part) and **(b)** gw−cath gap (`InsertionLengthRelative`, exists unused). If absolute depth must stay, normalize by *this route's* `path_context.total_length`, not the 900 mm device constant. - `guidance[0]`: add/substitute remaining arclength in mm clipped ÷ 400 so the scale is honest; keep the fraction only if you want a bounded progress bar. - `guidance[13:16]`: remove (dead), freeing 4 dims for the B proposals.

B. What is missing for path-cued navigation

The current "preview" is thin: one scalar max-curvature over the next 20 mm (`localguidance.py:546-558`), the tangent *at the current projection only*, scalar distances to next bifurcation/daughter, and a straight-line unit direction to the next daughter entry. The policy cannot see the *shape* of the corridor ahead or the geometry of the next take-off.

B1. Multi-point path preview (highest value). K upcoming path points at fixed arclength offsets (e.g. Δ ∈ {10, 20, 40, 80} mm), each expressed as `(p(s+Δ) − tip)` rotated to image 2D and divided by Δ. - Already computed: `LocalGuidance` caches `_polyline`/`_cumlen` at reset (:256-267) and `proj.s` per step (:298-306); arclength→point interpolation is exactly the `searchsorted` pattern of `PathProjectionCache.get_planned_path_tangent_at` (`pathcontext.py:991-1015`); the delta→image-2D rotation is `_entry_direction` (`localguidance.py:173-194`). - Bounding is free: chord ≤ arc, and tip is within `cross_track` of `p(s)`, so `|p(s+Δ)−tip|/Δ ≤ 1 + cross_track/Δ` — clip to [-1,1] after dividing by `Δ + 50`. - Cost: ~25 lines in `LocalGuidance.step`, +2K dims. This directly replaces what the policy currently gets from absolute tracking coordinates ("I know this bend is coming because I recognize where I am") with transferable "the path bends left in 20 mm".

B2. Junction geometry preview. At the next on-path junction: (i) 2D take-off direction of the daughter ostium, (ii) cos of the angle between continuing corridor and daughter take-off, (iii) signed in-plane angle. - Already computed: junction list with branch indices `_path_branch_sequence_with_junctions` (`pathcontext.py:328-339`, built at reset); off-path sisters per junction `_junction_off_path_candidates`; interior points 15 mm along a branch from a junction `_branch_interior_point` (`pathcontext.py:174-215`) — the exact primitive needed for take-off vectors; continuing-corridor tangent via `get_planned_path_tangent_at(j_arc + ε)`. - All static per episode → precompute per junction at reset, index by `proj.s` per step. ~40 lines in pathcontext + 4 dims in LocalGuidance. This is the feature that lets a policy learn "steep-angled ostia need the J-tip pre-rotated" as a *rule* instead of a per-mesh habit.

B3. Local vessel radius now / ahead. `get_local_radius()` exists and is already used every step by the state machine (`pathcontext.py:1040-1065`; radii loaded from `.mrk.json` "Radius" measurements, `dualdevicenav.py:184-194`; per-branch arrays `_branch_radii` built at reset). Radius *ahead* at `s+Δ`: `_branch_at_arclength(s+Δ)` (`pathcontext.py:1072-1081`) → nearest branch-polyline point → radius. Normalize ÷12 (`MAX_RADIUS_CEILING_MM`). Also expose the **clearance margin** `cross_track / get_local_tolerance()` (`pathcontext.py:1067-1070`) — a dimensionless, radius-aware version of feature 1 that transfers across vessel calibers far better than raw mm. 3 dims total.

B4. Distance-to-next-junction — already covered (features 6 and 11); no change needed beyond keeping both.

C. What is missing for buckle/stall awareness

The env computes rich buckle signals per step and shows the policy almost none of them:

Signal	Where computed	Exposed?
`delta_gw` — executed gw insertion this step	env5.py:832-835	**No**
`delta_s` — tip arclength progress this step	env5.py:838-841	**No** (guidance has no Memory; feature 0 is single-frame so the network cannot difference it)
fold condition `delta_gw ≥ 0.5 ∧ delta_s < 0.5 ∧ ¬d_corr_improving`	env5.py:858-865 (constants env5.py:92-94)	only as the **quantized counter** feature 21 (÷20) — the policy learns it is stalling only after the counter starts ratcheting
`d_corr_improving`	env5.py:849-856	No
commanded vs masked action: `last_cmd_action`, `last_exec_action`, `last_mask_reasons` (below_zero / max_length / tree_end)	`monoplanestatic.py:94-130`	**No** — `LastAction` reads `last_action` = clipped command *pre-masking* (`lastaction.py:18`, `monoplanestatic.py:86,92`), so the policy cannot tell its push was zeroed at tree end
gw−cath relative extension (gap)	derivable from both `inserted_lengths` values; `InsertionLengthRelative` exists (`insertionlengthrelative.py:7-40`)	only implicitly (network must subtract two ÷900-normalized scalars — poor conditioning)
catheter tip position / shape	`fluoroscopy.device_trackings2d` computed every access (`trackingonly.py:92-116`); `TrackingDevice2D` obs exists (`trackingdevice2d.py`)	**No** — `Tracking2D` uses combined `dof_positions`, the catheter tip is invisible in the union shape
accumulated device rotations	`monoplanestatic.py:59-61`, logged env5.py:1196-1199; `Rotations` obs (sin/cos) exists (`rotation.py`)	**No** — torsion state affecting J-tip response is unobserved (bend_hat 17-18 covers current in-plane bend only)
local radius / tolerance	`pathcontext.py:1040-1070`, logged env5.py:1160-1169	No

Proposed buckle block (5-6 dims, all mesh-invariant): 1. `slip = clip((delta_gw − delta_s)/4, −1, 1)` — continuous commanded-vs-achieved discrepancy (4 mm = max per-step motion at 30 mm/s ÷ 7.5 Hz). Mirror `prev_inserted_gw` / `prev_tip_s` onto the intervention exactly as the counters already are (env5.py:806-814), or keep prev-state inside LocalGuidance. 2. `gw_slack_norm = clip((inserted_gw − proj.s)/50, 0, 1)` — **the single most informative buckle scalar**: total wire length stored in bowing, the integral of every past fold. Both operands exist per step (`device_lengths_inserted[0]`, `get_projection().s`). 3. `gap_norm` — `InsertionLengthRelative(device_idx=0, relative_to_device_idx=1)` ÷ ~150 mm: guidewire extension beyond catheter, the key coordination variable for a two-device system. 4. `action_masked` — 1 or 2 dims: `float(last_cmd_action[i,0] != last_exec_action[i,0])` per device, from `monoplanestatic.py:95,129`. 5. Optional: catheter-tip → gw-tip 2D offset ÷ 150 (from `device_trackings2d[1][0]` and `[0][0]`), giving the policy the catheter tip explicitly. 6. Optional: `sin/cos(device_rotations)` via the existing `Rotations` obs (4 dims).

D. Wire-shape representation

What tracking encodes today: 10 points resampled at fixed 15 mm arclength spacing starting from the tip and walking proximally (`tracking2d.py:36-57`) — so 135 mm of distal wire (env5.py:243 says "150 mm"; it is 9×15 = 135), tail-padded by repeating the last point when less wire is inserted (the padding count itself leaks insertion depth). Points are absolute image-2D, then bbox-affine-normalized (anisotropically), then 2-frame stacked (`Memory`, FILL) — stacking happens *after* normalization. 40 dims.

Tip-relative is nearly a drop-in. The wrapper already exists: `RelativeToFirstRow` (`eve\eve\observation\wrapper\relativetofirstrow.py:8-43`) subtracts row 0 from rows 1..N (row 0 stays absolute). Proposed chain:

```
Tracking2D(n_points=10, resolution=15)
  → RelativeToFirstRow          # rows 1..9 = offsets from tip
  → NormalizeCustom(low=-135, high=+135)   # fixed mm scale; wrapper exists (normalizecustom.py)
  → Memory(n_steps=2, FILL)
```

Offsets are hard-bounded: $|p_i - p_0| \le 15 \cdot i \le 135$ mm, so the fixed scale is exact and **identical for every mesh** — the mesh bbox disappears from the observation entirely. Two consequences to handle: 1. **Row 0 stays absolute** in `RelativeToFirstRow` — either zero it (subclass, 3 lines) or deliberately repurpose it as the per-step tip displacement $tip_t - tip_{t-1}$ (÷4 mm), which restores the velocity information otherwise weakened by making both stacked frames tip-relative (frame-differencing tip-relative shapes yields shape-change but loses rigid tip translation). 2. Keep the offsets in the fixed image axes (do *not* rotate into a tip-heading frame): the action's rotation/translation semantics are defined relative to the C-arm view, and heading is already supplied by features 2-4; a per-step rotating frame would inject rotational noise into every point.

This change also fixes the anisotropic-affine distortion problem for free (single isotropic scale), so wire curvature reads identically on every mesh.

Verdict on D: yes — tip-relative trailing points at fixed mm scale is a drop-in improvement (two existing wrappers + one small subclass), removes the largest memorization channel (A.1), and preserves everything the policy legitimately needs from wire shape (bend, J-tip pose, buckle loops), which are all translation-invariant properties.

Priority order for the multi-mesh experiment

1. Tip-relative tracking (D) — kills leak A.1.
2. Drop absolute `target`, rely on guidance 0/8/9/10 (+ optional clipped tip→target offset) — kills A.2.
3. Replace `inserted_lengths` with slack + gap (A.3 + C.2/C.3).
4. Add path preview K-points (B1) and junction take-off geometry (B2) — gives the policy something transferable to navigate *by* once the memorization crutches are gone.
5. Add slip + action-mask buckle features (C.1/C.4); radius/clearance features (B3).
6. Delete dead phase one-hots 13-16; fix the "28-dim" docstring; verify `image_rot_zx` and mesh canonical orientation are held constant across all training/test meshes (`dualdevicenav.py:137-142`,

`DualDeviceNavCustom` `fluoroscopy_rot_zx` / `rotation_yzx_deg` args).

PART 3 OF 4

Asymmetric Actor-Critic + Recovery-Behavior Training — Code-Grounded Design Map

Agent: Privileged-signal plumbing audit · Source file: 8.md

Policy obs today (BenchEnv5, `training _scripts/util/env5.py:242-301`): `ObsDict{tracking (10pt×2D×2 frames=40), target (2), last_action (4), guidance (30), inserted_lengths (2)}` ≈ 78 flat dims. Everything below is relative to that.

1. Privileged signal inventory

`save_checkpoint` fields confirmed — `eve/eve/intervention/simulation/sofabeamadapter.py:123-159` captures `xtip`, `rotation_instrument`, `index_first_node`, `dof_positions` plus `_DOF_EXTRA_FIELDS` (`sofabeamadapter.py:114-121`): `velocity`, `force`, `externalForce`, `free_position`, `free_velocity`, `derivX` — each read via `getattr(ic.DOFs, name).value`, silently skipped if the build lacks it. All are equally readable **per step**, not just at checkpoint time.

Signal	Source / access path	Per-step cost	Value-for-critic hypothesis
Full beam node positions + **quaternions** (n_nodes×7)	`ic.DOFs.position.value` — only xyz, reversed, kept as `dof_positions` (`sofabeamadapter.py:319-326`); quats discarded	numpy copy, µs	Node orientations encode torsional windup / twist along the wire — the hidden state behind buckling and rotate-to-unbuckle.
Node velocities (n_nodes×6)	`ic.DOFs.velocity.value`	µs	True kinematic + angular velocity. Policy only gets a 2-frame (133 ms) position finite-difference (`env5.py:242-260`); critic gets exact dynamics → tighter TD targets.
Node forces	`ic.DOFs.force.value`	µs	Internal elastic + solved contact resultant. Large localized force = wedge/fold load *before* it appears kinematically. Strongest single privileged feature for fold-stall value prediction.
External forces	`ic.DOFs.externalForce.value`	µs	Complements `force` .
Constraint correction / contact proxy	`ic.DOFs.position.value − ic.DOFs.free_position.value` (and velocity analog)	µs	Free-motion minus constrained state = per-node contact impulse without touching SOFA's contact-pair API (scene uses LCP + FrictionContactConstraint, `sofabeamadapter.py:349-371` ; enumerating contact pairs would need narrow-phase queries — the free_* delta is the cheap proxy).
Solver derivative	`ic.DOFs.derivX.value`	µs	Marginal; include only because it's free.
Inserted lengths	`ic.m_ircontroller.xtip.value` → `inserted_lengths` (`sofabeamadapter.py:327-329`)	cached	Already in policy obs.
Instrument rotations	`ic.m_ircontroller.rotationInstrument.value` → `rotations` (`sofabeamadapter.py:330-332` , `monoplanestatic.py:59-61`)	cached	**Not in policy obs** (only STEP logs, `env5.py:1196-1201`). Absolute roll of both devices — key for rotate-to-unbuckle credit assignment.
Active beam node index	`ic.m_ircontroller.indexFirstNode.value`	µs	How much wire is mechanically simulated; mild.
Collision-bead positions	`beam_collis.CollisionDOFs` MechanicalObject (`sofabeamadapter.py:526-539`)	µs	Redundant with DOFs positions; skip.
Full 3D wire tracking	`fluoroscopy.tracking3d` (`eve/eve/intervention/fluoroscopy/trackingonly.py:78-89`)	already computed per step	Policy sees 10 pts, 2D-projected, normalized. Full 3D shape (out-of-plane component) is privileged vs a monoplane deployment.

Signal	Source / access path	Per-step cost	Value-for-critic hypothesis
Per-device trackings	`fluoroscopy.device_trackings3d` (`trackingonly.py:91-110`) — exists, used only by `snapshot.py:96`	O(n_nodes) numpy, µs	Splits combined tracking into guidewire vs catheter polylines by inserted length. Policy currently **cannot distinguish catheter shape from guidewire shape** — catheter-support state is central to fold recovery.
Physical branch tag	`classify_physical_branch(intervention)` (`eve/eve/util/pathcontext.py:83-132`)	projects tip on every branch, sub-ms (already run per tick in `MultiTargetEnv5.step`, `env5.py:2052-2055`)	Target-independent ground-truth "which vessel am I in" as one-hot.

Suggested privileged vector (compact, ~O(30-60) dims): tip-region node velocities (last k nodes), per-node force magnitude summary (max + mean + argmax-arclength), free_position delta magnitude summary, both device rotations, catheter tip position + gw-tip↔cath-tip separation from `device_trackings3d`, physical-branch one-hot, plus the env-computed scalars in §2.

2. Signals the env computes but hides from the policy

Signal	Where computed	In `info`?	In obs?
Fold detector inputs: `delta_gw` (commanded insert), `delta_s` (tip arclength delta), `d_corr_improving`	`env5.py:832-855`	No	No (constituents partially derivable)
`_fold_stall_count` / truncation at `FOLD_STALL_STEPS=20`	`env5.py:858-870`; constants `env5.py:92-94`	No (only aggregate `grader_failure_timeout`, `env5.py:1359-1363`)	Normalized only, feature 21 via intervention mirror `env5.py:806-814` → `localguidance.py:402-407,489`
`_off_branch_steps` / timeout at `OFF_BRANCH_GRACE_STEPS=50`	`env5.py:1015-1064`; constant `env5.py:71`	No	Normalized, feature 20 (`localguidance.py:488`)
Off-path arc since divergence	`pathcontext.py:1294-1332` (`_capture_divergence_point`, `get_off_path_arc_since_divergence`)	No	Feature 19, clipped 50 mm (`localguidance.py:36-38,94`)
`classify_physical_branch`	`pathcontext.py:83-132`	Yes — `info["final_branch_short"]` `env5.py:1389`, broadcast `env5.py:2079`	No
Cross-track dist + radius-aware tolerance + **excess**	raw: `pathcontext.get_projection().cross_track_dist` (`pathcontext.py:504-520`); tolerance `pathcontext.py:1040-1070`; excess used in reward `eve/eve/reward/arclengthprogress.py:171-178`	No	raw clipped/50 as feature 1; **excess & tolerance hidden**
Wrong-commit latch / commit events	`pathcontext.py:353-365, 1217-1239, 1244-1273` (`_committed_forks`, `_wrong_commit_fired`, `_daughter_commit_events`)	Partially: `received_correct/wrong_daughter`, `reached_target_daughter` (`env5.py:1338-1340`)	counts as features 23-24; latch itself hidden
Cath-vs-gw inserted gap	both lengths `monoplanestatic.py:55-57`	No	Both in obs but normalized by *different* max lengths →

Signal	Where computed	In `info`?	In obs?
			mm gap not linearly recoverable
Commanded vs executed action	`monoplanestatic.py:94-130`: `last_cmd_action` (:95), `last_exec_action` (:129), `last_mask_reasons` (:130); masks = below_zero/max_length/tree_end (:104-126)	No (mask reasons only in heuristic STEP logs `env5.py:1178-1180`)	`LastAction` obs reads `intervention.last_action` = clipped **pre-mask** command (`monoplanestatic.py:86-92`, `eve/eve/observation/lastaction.py:17-18`) — the executed action is recorded but never surfaced
Grader outcome flags	`env5.py:1337-1392`	Yes	No

Note: `Episode.infos` are dropped at `to_replay()` (`eve_rl/eve_rl/replaybuffer/replaybuffer.py:61-84`) — info-carried signals **never reach the replay buffer**. That drives the plumbing choice below.

3. Minimal-diff privileged-critic plan

Key discovery — obs-side is far cheaper than info-side. The buffer stores only `flat_obs` (`pervanillastep.py:158`, `experience_cache.py:229-234`), and flattening is a plain dict-order concat (`single.py:105,118,179,224` → `eve_rl/eve_rl/util/flattenobs.py:5-35`). So: append a `"privileged"` key **last** in the ObsDict, let it ride inside `flat_obs` everywhere (buffer, PER, npz harvest, offline AWAC/IQL bulk-import — all zero-change), and make **the policy slice it off internally**. Critics consume full width.

The Batch-NamedTuple precedent you asked about is `replaybuffer.py:120-133` (`is_weights`/`indices` appended with `None` defaults, threaded through `Batch.to()` :135-178 and accessed by name in `sac.py:256-279`) — that's the template for the info-side alternative (Option B), which would additionally touch `Episode.add_transition`/`to_replay`, `PERVanillaStep.push/sample/export_all/import_all/bulk_import_arrays`, `experience_cache`, and `sac.update` (~7 files). Option A needs 3:

1. **New** `eve/eve/observation/privileged.py` — an `Observation` subclass reading `intervention.simulation._instruments_combined.DOFs.*`, `intervention.fluoroscopy.device_trackings3d`, `intervention.device_rotations`, plus the env-counter mirrors that already exist on the intervention (`_env_off_branch_steps` etc., set in `env5.py:799-814` — LocalGuidance precedent `localguidance.py:402-407`). Normalize internally (fixed scales), define `space` so `agent.py`'s obs-space sampling sizes it.
2. `training _scripts/util/env5.py:293-301` — add `"privileged": priv_obs` as the **last** ObsDict entry (dict order = flat layout tail). Also mirror `path_context` extras if desired (pattern at :806-814).
3. `eve_rl/eve_rl/network/gaussianpolicy.py:38-54` — 2-line change: `obs_batch = obs_batch[..., :self.n_observations]` at the top of `forward` and `forward_play`. `log_prob` (:56-76) calls `self.forward` → covered. SAC's `_update_policy`, `_get_update_action`, `_get_expected_q` (`sac.py:400-609`) pass full-width `states` to both policy and critics → **zero sac.py changes**; target critics are deepcopies of q1/q2 (`sacmodel.py:58-61`) → widened automatically.

4. `training _scripts/util/agent.py` (`:28-33, 44-74`; repeat for the AWAC/IQL class ~:179-241 and function ~:381-422) — compute `n_obs_total` from the obs space, `n_obs_policy = n_obs_total - priv_dim`; pass `n_obs_policy` to `GaussianPolicy`, `n_obs_total` to both `QNetwork`s (and `VNetwork` at :241 for IQL). **Trap:** one `q1_embedder` object is currently shared across q1, q2, *and* policy (`agent.py:48,60,72-73`); `MLP.n_inputs`/LSTM setters raise on mismatched re-set (`mlp.py:40-46`). The policy needs its **own** embedder instance sized `n_obs_policy`.
5. `qnetwork.py` **— no change** (`head.n_inputs = n_observations`, body input = head_out + actions, :24-27, 33-39).
6. **Play-only workers — confirmed unaffected**: workers are built with `algo.to_play_only()` (`synchron.py:701` train, `:232` eval-restart) → `SACPlayOnly` → `SACModelPlayOnly` holding *only* `policy` (`sacmodel.py:9-27, 152-155`). No critic is ever constructed in a worker. Workers run deepcopied `BenchEnv5`, so they can compute privileged obs (it's sim-side "deployability", not process-side, that's restricted).

Residual costs: old checkpoints/replay npzs have 78-dim obs → fresh harvest or a pad-on-load shim in `buffer_filter`/`experience_cache`; probe states (`util/probe_evaluator.py`) need regeneration; `ConfigHandler` env serialization must tolerate the new component (same pattern as existing observations).

4. Recovery-behavior training assessment

(a) Where truncations fire and what relaxing them implies. Fold-stall: counter `env5.py:858-865`, truncate at :867-870 (`FOLD_STALL_STEPS=20`). Off-path: increment :1015-1017, truncate :1043-1062 (`OFF_BRANCH_GRACE_STEPS=50`), with OST overshoot softening :1051-1059 and geometric on-path correction :1004-1014. Penalty −5/−1 applied :1074-1087. Then the **grounding rewrite** `truncated → terminated` at `env5.py:1394-1423`, whose comment explicitly assumes failures are *absorbing*. Recovery training breaks that assumption: if buckled/off-path states are recoverable, storing `done=True` there teaches the critic V(stuck)=penalty-and-nothing-after — the exact opposite of what the recovery policy needs. But the buffer cannot bootstrap-through-truncation either: `EpisodeReplay` keeps only `terminals` (`replaybuffer.py:78-84`), truncations are discarded. Cleanest configuration: **delete the fold/off-path truncations (keep the counters as obs/privileged features), let MaxSteps(600) end every failed episode, keep the rewrite so max_steps grounds the value**. The −5 anchor disappears; the existing shaped signal already prices recovery correctly — `ArcLengthProgress` off-path shaping is symmetric (retract toward divergence point earns *positive* reward, `arclengthprogress.py:117-157`), plus −0.007/step off-path segment reward (`env5.py:1548-1601`) makes dwelling expensive. Episode wall-time roughly doubles-to-triples for failures (20-50-step aborts → up to 600 steps) — the main cost. Also update: `localguidance.py:96-97` normalization constants, `grader_failure_timeout`/`is_clean` definitions (`env5.py:497-506, 1359-1363`; `experience_cache.py:160-169`) which key on the thresholds, and heuristic-mode aborts (leave heuristic path gated on `_heuristic_mode`, as OST already does).

(b) Stuck-state restore pool — feasible, pattern already exists twice. Capture: clone `_save_rva_checkpoint` (`env5.py:1668-1734`) / pre-bif11 memo (`env5.py:924-963`) into a buckle-harvester: when `self._fold_stall_count` first reaches N (e.g. 10, before the 20-step kill) or `_off_branch_steps` reaches M, call `sim.save_checkpoint()` + `tracking3d` + sidecar (target branch, fold/off counts, `proj_s`), write to `STUCK_CHECKPOINT_DIR` unconditionally (unlike pre-bif11's write-on-success gate :1425-1448 — here failures are the product). ~40 LOC, env-var gated. Restore: `CheckpointRestoreWrapper` (`checkpoint_restore.py:30-`

133) already injects `options["restore_checkpoint"]` per reset; `BenchEnv5.reset` applies it **after** `super().reset()` (`env5.py:604-660`) — required because `InsertionPoint.start.reset()` would zero the wire (`sofabeamadapter.py:184-192`, `monoplanestatic.py:149-153`). Known pitfalls, all already handled in code: **double-restore first-restore quirk** (`env5.py:637-642` — first restore after scene build drops `dof_positions`; applied twice via `_restore_warmed_up`), **settle steps** (default 3 with fuller-save vs 50 legacy, `sofabeamadapter.py:233-238`), **rotation zeroing for old-format checkpoints** (`sofabeamadapter.py:206-216`), obs/reward re-reset + `path_context.invalidate()` (`env5.py:646-659`), pre-latching already-passed junctions (`env5.py:690-723`). Per-restore cost ≈ npz load + `Simulation.reset` + field assigns + 3 animates; one env step is ~22 animates (`duration=1/7.5 s`, `dt=0.006`, `sofabeamadapter.py:74-76`), so settle ≈ 0.14 env-steps — negligible vs the reset you pay anyway. Two gaps to fix: (i) the wrapper ignores the sidecar's `target_branch` — fine for per-daughter runs with `default_target_branch`, otherwise inject `options["target_branch"]` from the json; (ii) counters reset to 0 on reset (`env5.py:566-570`), which conveniently gives the restored buckled wire full grace — but if truncations are kept, deep-wrong-branch restores may need the existing `_heur_suppress_wrong_branch`-style bypass (`env5.py:1042`).

(c) Heuristic recovery demos — yes, three distinct behaviors exist. (1) Generic off-path retract: `heuristic_policy.py:112-129` — random `gw_trans ∈ [-10,-1]` with catheter following whenever `is_on_correct_path()` is false. (2) LCCA stuck-recovery: `heuristic_policy_lcca.py:70-88` (retract-catheter-while-pushing-wire and elastic "burst" strategies), stuck detection `_is_stuck` :240-252 (10-step tip window, <1 mm). (3) C8 stagnation-retract variants in RCCA/RVA/LVA policies (`heuristic_policy_rcca.py:118-124, 397-403` and twins) and heuristic_controller Phase-1 pure retract (`heuristic_controller.py:178-259`). Caveat: heuristic episodes abort on the same 50-step timeout (`_heuristic_abort`, `env5.py:1060-1061`), so recorded demos contain only recoveries that *succeed within grace* — relax the timeout in heuristic-demo harvesting mode too if you want deep-recovery demos. Demos enter the buffer with `is_demo=True` → PER demo bonus (`single.py:600-608`, `pervanillastep.py:135-145`).

5. Auxiliary-head feasibility — easy, ~4 small edits

`MLP` already supports multiple parallel output heads: `output_layer_size` as a list builds N linear heads off the last hidden layer (`mlp.py:62-76`) and `forward` returns a list (:83-92) — exactly how GaussianPolicy gets `[n_actions, n_actions]` (`gaussianpolicy.py:32`). Add `n_aux` to `GaussianPolicy.__init__` → `body.output_layer_size = [n_actions, n_actions, n_aux]`; `forward` unpacks 3, returns `(mean, log_std)` and stashes `self._last_aux` (keeps every caller — `sac.py:400,477,596`, play paths — signature-stable). Labels: with the §3 privileged tail in `flat_obs`, labels are already in the batch — slice `states[..., n_obs_policy + k]` (buckle flag = fold-counter feature > 0; contact-force magnitude feature). Loss hook: end of `SAC._update_policy` (`sac.py:400-506`), before `policy_loss.backward()` at :490-491: `policy_loss += aux_coef * F.mse_loss(self.model.policy._last_aux_from(states), labels)`. No separate optimizer needed — the aux head's parameters live in `body._output_layers` (an `nn.ModuleList`), so `policy_optimizer = Adam(policy_base,...)` (`agent.py:75-78`) already covers them. Watch-outs: old policy checkpoints won't `load_state_dict` strictly (extra head keys) — warmstart needs `strict=False`; play-only deepcopy carries the head harmlessly (extra head computed and ignored in `forward_play`). Hindsight labels ("will fold within k steps") would instead require Episode-level relabeling at `push()` — that escalates to Option-B plumbing; the predict-current-privileged variant is the minimal, still-useful representation-shaping version.

PART 4 OF 4

Design: From Mesh Memorization to Path-Cued, Recovery-Capable Navigation

Agent: Final synthesis (main session) · Source file: 9.md

The headline good news, in three parts: (1) your Plan-v12 de-hardcoding already made the reward core and path state-machine mesh-portable — they derive everything from the loaded vessel tree at reset; (2) a config-swappable mesh loader (DualDeviceNavCustom) and a per-episode geometry-randomization precedent (ArchVariety/AorticArchRandom) already exist in the codebase; (3) the memorization problem is localized and fixable — it lives in three observation components and two odometer features, not diffusely through the design. The work is real but it's surgery, not a rewrite.

1. Why the policy memorizes the mesh today

The audit found the leaks are ranked and specific:

The 40-dim wire tracking is the dominant leak. Tracking2D produces absolute image-plane coordinates, then NormalizeTracking2DEpisode scales them by the mesh's own centerline bounding box (`trackingonly.py:53-76`). Every frame tells the policy where in this mesh's bbox it is — absolute tip position becomes a progress state, which is exactly what enables open-loop "smooth direct traversal." Worse, x and z scale by different factors, so wire angles and curvature are distorted by a per-mesh-different anisotropic affine.

The target is an absolute 2D coordinate in the same mesh-bbox normalization. On one mesh the four daughters sit in four recognizable coordinate clusters — the policy can key its entire route off this constant instead of reading path guidance. Across meshes it's a mesh-identity fingerprint.

`inserted_lengths` ÷ 900 is a per-mesh odometer. With a fixed insertion point, mesh landmarks live at fixed insertion depths — perfect support for memorized action schedules, meaningless on a new mesh.

`d_rem_norm` (guidance feature 0) is a route-fraction index — "at f=0.4, hook left" is memorizable, and the same value means different millimeters on different meshes.

Guidance features 13–16 (phase one-hots) are dead in RL — they're written by the heuristic and read a constant default during RL — and their semantics are anatomy-bound anyway. Free dims to reclaim.

Meanwhile most of the 30-dim guidance block (cross-track, tangent, heading error, routed distance-to-entry, entry direction, topological flags, off-path arc) is honestly path-relative and transferable — the transferable skeleton already exists; it's being drowned out by the absolute-position crutches, which are strictly easier for the network to exploit on a single mesh.

And one structural point: even with perfect observations, single-mesh training never forces closed-loop control — with fixed geometry and near-deterministic dynamics, a memorized schedule is always a valid solution. Multi-mesh training itself is the strongest regularizer you can add; the observation redesign is what makes it possible for the policy to find the transferable solution instead of just failing.

2. The observation redesign (the centerpiece)

Everything below uses primitives that already exist — I list what each change reuses.

Remove / replace (kill the leaks):

Today	Replace with	How
Absolute bbox-normalized tracking	Tip-relative wire shape at fixed mm scale	Existing wrappers: Tracking2D → RelativeToFirstRow → NormalizeCustom(±135mm) → Memory(2). Offsets are hard-bounded (
Absolute 2D target	Delete. Routing content already lives in guidance 0/8/9/10	Optionally add a clipped tip→target offset (±50mm ÷ 50) so near-target homing stays sharp; far targets saturate and path guidance does the routing.
`inserted_lengths` ÷ 900	`gw_slack` + gw−cath gap	`gw_slack` = clip((`inserted_gw` − proj.s)/50, 0, 1) — inserted length minus tip arclength = wire stored in bowing, the single most informative buckle scalar, and mesh-relative by construction. Gap via the existing-but-unused InsertionLengthRelative.
`d_rem_norm` fraction	remaining arclength in mm, clipped ÷ 400	honest scale across meshes; keep the fraction too if you want a bounded progress bar.
Phase one-hots 13–16	delete	frees 4 dims.

Add — path preview (what "take cues from the planned path" means concretely):

B1 — K-point lookahead (highest value): planned-path points at arclength offsets Δ ∈ {10, 20, 40, 80}mm ahead, each expressed as (p(s+Δ) − tip)/(Δ+50) in the fixed image frame. This replaces "I recognize where I am, so I know the bend is coming" with "the path bends left in 20mm" — the transferable cue. ~25 lines in LocalGuidance.step; the arclength→point interpolation and delta→2D rotation primitives already exist (`pathcontext.py:991-1015` , `_entry_direction`).

B2 — junction take-off geometry: at the next on-path junction, the daughter ostium's 2D take-off direction plus the angle between continuing corridor and daughter take-off. Precomputable per junction at reset from `_path_branch_sequence_with_junctions` + `_branch_interior_point` . This is the feature that lets a policy learn "steep-angled ostia need the J-tip pre-rotated" as a rule instead of a per-mesh habit — the generalized version of the LCCA hook.

B3 — radius now/ahead + clearance margin: `get_local_radius()` is already computed every step by the state machine; expose it, plus radius at s+Δ, plus the dimensionless `cross_track` / `get_local_tolerance()` — a caliber-aware clearance that transfers across vessel sizes far better than raw mm.

Add — buckle/stall awareness (what "know to retract" needs): the env already computes all of this per step and hides it from the policy:

slip = clip((`delta_gw` − `delta_s`)/4, −1, 1) — commanded-vs-achieved discrepancy, the continuous version of the fold detector's own inputs (`env5.py:832-855`). Today the policy only sees the quantized counter after it starts ratcheting; give it the raw signal so it can react at step 1.

Action-masked flags: MonoPlaneStatic records commanded vs executed actions and mask reasons (`monoplanestatic.py:94-130`) but LastAction reads the pre-mask command — the policy literally cannot tell its push was zeroed at the tree end. Expose cmd ≠ exec per device.

Catheter tip offset: fluoroscopy. `device_trackings3d` splits guidewire/catheter polylines and is used only by the snapshot tool. Add catheter-tip→gw-tip 2D offset (and optionally 2-3 catheter shape points) — this closes

the "catheter steered semi-blind" gap from the first review.

Optional: sin/cos of accumulated device rotations (the Rotations obs class exists) — torsion state that the J-tip response depends on.

One invariance rule to enforce: all direction features are expressed in the fixed C-arm image frame, which is only mesh-invariant if every mesh is loaded in a consistent anatomical orientation with the same `image_rot_zx`. Pin `image_rot_zx` =[20,5] and `rotation_yzx_deg` conventions across all train/test meshes — this is a mesh-preparation checklist item, not a code change.

3. Learned recovery behaviors (retract, unbuckle, re-approach)

The blocking problem: you can't learn recovery from states the env kills you for entering. Fold-stall truncates at 20 steps, off-path at 50, both with −5 and terminated=True — teaching the critic V(buckled) = penalty-and-nothing-after, the exact opposite of "buckled states are recoverable with the right maneuver." Three coordinated changes:

Stop truncating on fold-stall and off-path; keep the counters as observation features. Let MaxSteps(600) end failed episodes; keep the grounding rewrite at `max_steps` (and add a time-remaining feature so the value function can represent it honestly). The reward already prices recovery correctly without the −5 anchor: the off-path shaping is symmetric — retracting toward the divergence point earns positive reward (`arclengthprogress.py:117-157`) — and per my earlier review, drop the non-telescoping −0.007/step off-path tax so a successful recovery excursion isn't net-negative. Cost: failure episodes run 2–3× longer in wall-clock. Which is why you also want:

A stuck-state restore pool — recovery curriculum from exactly the states that matter. The pattern exists twice already (pre-bif11 and RVA checkpoint harvesters, `env5.py:924-963`): snapshot `save_checkpoint()` when the fold counter first hits ~10 (before failure is hopeless) or off-path hits ~25, write to a `STUCK_CHECKPOINT_DIR` pool with a sidecar (target branch, counters, arclength). Then train dedicated episodes that start buckled via the existing CheckpointRestoreWrapper — short episodes, dense recovery signal, no wall-clock waste walking into trouble. All the known restore pitfalls (first-restore double-apply, settle steps, re-latching passed junctions) are already handled in code. This is your proven restore-at-fork trick (+53pp on RCCA) re-aimed at recovery skills.

Recovery demos exist in the heuristics — harvest them. Three distinct behaviors: the generic off-path retract (`heuristic_policy.py:112-129`), the LCCA stuck-recovery with catheter-retract-while-pushing and elastic-burst strategies, and the C8 stagnation-retract variants. Caveat: heuristic episodes currently abort on the same 50-step timeout, so recorded demos only contain fast recoveries — relax the timeout in demo-harvest mode to capture deep recoveries. Demos land in the clean lane / `is_demo` stream that AWAC already consumes.

Update the dependent definitions when you do this: the LocalGuidance counter normalizations, `grader_failure_timeout`, and `is_clean` all key on the old thresholds.

4. The privileged critic — "SOFA knows everything," concretely

What the critic should see (the privileged vector, ~30–60 dims):

Signal	Source	Why it helps
Node forces (max, mean, argmax-arclength)	ic.DOFs.force.value — µs numpy read	Wedge/fold load appears in forces before it appears kinematically — the leading indicator for fold-stall value prediction
Contact proxy	position − `free_position` per node	Per-node contact impulse without touching SOFA's contact-pair API
Node velocities (tip-region)	ic.DOFs.velocity.value	True dynamics vs. the policy's 133ms finite-difference — tighter TD targets
Device rotations	ic.`m_ircontroller` .rotationInstrument	Torsional windup — the hidden state behind rotate-to-unbuckle
Catheter tip pose + gw/cath separation	`device_trackings3d`	Full two-device state
Physical branch one-hot	`classify_physical_branch`	Ground-truth "which vessel am I actually in," target-independent
Fold/off-path counters, slip, cross-track excess	already computed in env5/pathcontext	The env's own hidden diagnosis

The plumbing trick that makes this cheap (Option A — the audit's key finding): don't route privileged state through the info dict (that would touch Episode, buffer, PER, caches, ~7 files, because Episode.infos are dropped at `to_replay()`). Instead, append a "privileged" key as the last entry of the ObsDict — it rides inside `flat_obs` through the buffer, PER, npz caches, and offline tooling with zero plumbing changes — and make the policy slice it off internally:

New `eve/eve/observation/privileged.py` — reads the SOFA fields + intervention mirrors, normalizes with fixed scales.

`env5.py:293-301` — add "privileged" last in the ObsDict.

`gaussianpolicy.py:38-54` — two lines: `obs_batch` = `obs_batch`[..., :self.`n_observations`] at the top of forward/`forward_play` (`log_prob` calls forward, so it's covered).

`agent.py` — pass `n_obs_policy` to the policy, full width to both Q-nets. Trap (this is the J1 finding from the first review resurfacing): the same embedder object is currently shared across q1/q2/policy — the policy must get its own embedder instance sized to the policy width. This forces the fix the earlier review wanted anyway.

`sac.py` needs zero changes (it passes full-width states to both policy and critics; targets are deepcopies of the critics), and play-only workers are structurally unaffected — they never construct critics. AWAC benefits directly: the advantage Q(s,`a_buf`) − Q(s,a_π) is computed entirely by the (now privileged) critics, so demo-weighting quality improves without touching the policy's deployability.

Two force multipliers on top:

Auxiliary heads (cheap, do it): the MLP already supports multiple output heads (`output_layer_size` as a list — it's how the policy gets mean+`log_std`). Add a head predicting current privileged quantities (buckle flag, contact-force magnitude) from policy obs, supervised from the privileged tail already sitting in the batch, loss added before the policy backward. This shapes the policy's representation to infer contact state from deployable signals — exactly the inference "know when you're buckling" requires. No separate optimizer needed.

Teacher–student as diagnostic: optionally train a teacher policy on privileged obs too, then distill to the deployable-obs student. The distillation gap is a measurement of what your observation is missing — if the student matches the teacher, the obs redesign is sufficient; if not, the gap tells you where to look next.

5. Multi-mesh training mechanics

The critical path is mesh preparation, not RL code. The whole tag-matching stack (pathcontext daughters, env5 reward classes, target templates, heuristics) keys on the naming convention: uppercase Centerline curve - `LCCA/RCCA/RVA/LVA.mrk` daughters + numbered trunk/bridge curves. Your `vmr_processing_tools` pipeline produces the right format; the centerlines must be renamed to the convention (and radii verified) or the classification silently degrades to "other". Also reconcile the folder-name mismatch: DualDeviceNav reads `Centrelines_comb`, DualDeviceNavCustom reads Centrelines.

Per-worker fixed meshes — the recommended mechanism, near-zero marginal cost. Each worker already builds its SOFA scene exactly once at startup and never rebuilds (sofabeamadapter only reconstructs when `mesh_path` changes; FromMesh.reset() is a no-op). So the change is one site: replace deepcopy(self.`env_train`) at `synchron.py:702` with an `env_factory(i)`, and assign mesh k = i % 3 across the 16 workers via three DualDeviceNavCustom interventions. Forward the factory through BenchAgentSynchron, route per-mesh target schedules per worker, and re-derive `insertion_z` per mesh (345 is "40mm below bif2" on this mesh). Per-episode mesh swap is possible (the AorticArchRandom precedent) but pays a full scene rebuild per episode and needs an intervention-rebuild API — not worth it for 3 meshes.

Evaluation protocol: keep the held-out mesh as a dedicated `env_eval` (its own DualDeviceNavCustom, own target pool), don't split eval seeds across worker meshes. Report a per-mesh Quality vector + the held-out number; the train-mesh-average minus held-out gap is your generalization metric. Add a second eval track from the stuck-state pool: recovery success rate — that's the metric for the behaviors you're trying to buy.

Heuristics/demos per mesh: the base heuristic and controller are geometry-relative (zero retuning). RCCA/RVA need a junction-token-chain check + re-running the existing variant grid; LCCA needs ~4 constants + one direction vector; LVA is the risk — its strategy relies on this mesh's arch passively funneling the wire, which may simply not hold elsewhere. Demos must be regenerated per training mesh regardless (arclengths differ), which the recovery-demo harvesting (§3) folds into naturally. Note HER (from the first review) compounds here: per-mesh relabeling multiplies each mesh's scarce successes.

Constraint to respect: all meshes must produce identical obs/action shapes (network dims freeze from the master env) — same tracking point count, same guidance dims, same daughter count or padded one-hots.

6. Costs and compatibility breaks — be deliberate about these

Everything obs-related is a hard break: new obs dims invalidate all checkpoints, the seed buffer, and probe states. You were already breaking these with the v3 bug-fix package (MLP ReLU, reward fixes) — bundle all breaking changes into one generation so you pay the re-harvest exactly once.

Removing fail-fast truncations costs throughput (failures run to 600 steps). The stuck-pool curriculum claws most of it back; you can also keep a much looser off-path bound (e.g. 150 steps) as a backstop rather than none.

Multi-mesh will look worse before it looks better. Expect single-mesh Quality to drop when the memorization crutches are removed — that's the point. Judge the new generation on the per-mesh vector + held-out gap,

not on matching the old single-mesh curve.

7. Suggested sequence

Gen-3 foundation (one breaking change-set): v3 bug fixes (previous review) + observation redesign (§2) + privileged tail & critic widening (§4, it's ~4 file edits and the policy slice makes it forward-compatible) + aux heads. Validate on the current mesh — target: recover roughly current performance without the absolute-position features (that alone proves the policy went closed-loop).

Recovery training on the current mesh: truncation relaxation + stuck-pool harvest/curriculum + recovery-demo harvest (§3). Metric: recovery rate from the stuck pool. This de-risks the behavior question before multi-mesh multiplies everything.

Meshes: prepare 3+1 meshes to the naming convention (the true critical path — start it in parallel with 1–2); per-worker mesh assignment; per-mesh heuristic re-verification and demo generation; held-out eval env.

Train the generalist: 3-mesh training, held-out eval, per-mesh curriculum weighting by success rate if one mesh lags. Optionally add the teacher–student distillation pass as the observability audit.

The design intent, in one sentence: make the transferable path/recovery cues the easiest signals for the network to use (preview, clearance, slip, catheter state), make the memorizable ones unavailable (absolute position, odometers), give the critic the physics the policy can't see, and let the meshes themselves force closed-loop behavior. All of it is buildable with primitives already in your codebase — I've saved the full design map to memory, and I can start on any phase; the natural first bite is the observation redesign plus the privileged-tail plumbing, since they share the same breaking change-set.